فصل ۵: اشاره‌گر، آرایه و رشته

چرا این فصل؟ سخت‌ترین تست‌های «خروجی کد» در C معمولاً از همین فصل می‌آیند: `*p++` در برابر `(*p)++` ، حساب آرایهٔ دوبعدی، و `sizeof` در برابر `strlen` . **فرض‌ها:** `int` ۴ بایتی و اشاره‌گر ۸ بایتی (سیستم ۶۴ بیتی). **اعتبار:** خروجی کدها با `gcc` اجرا و چک شده است. **زمان مطالعه:** یک بلوک ۹۰ دقیقه‌ای.

درس ۱: اشاره‌گر

عبارت	معنی
`&x`	آدرس x
`*p`	مقداری که p به آن اشاره می‌کند
`int **pp = &p;`	اشاره‌گر به اشاره‌گر؛ `**pp` همان x است
`*p++`	یعنی `*(p++)` : **مقدار فعلی** را بده، بعد **اشاره‌گر** جلو برود
`(*p)++`	**مقدار** را یکی زیاد کن (اشاره‌گر ثابت می‌ماند)؛ مقدار قدیمی را برمی‌گرداند
`++*p`	مقدار را زیاد کن و مقدار **جدید** را برگردان

مثال: با `a = {10, 20, 30}` و `p = a` :

- `x = *p++` ⇐ `x = 10` و p حالا به ۲۰ اشاره می‌کند.
- `y = (*p)++` ⇐ `y = 20` و `a[1]` می‌شود ۲۱.
- `z = ++*p` ⇐ `z = 22` و `a[1]` می‌شود ۲۲.

درس ۲: آرایه و حساب اشاره‌گر

- `a[i]` دقیقاً همان `*(a + i)` است (و حتی `i[a]` هم درست است).
- `p + 1` به **عضو بعدی** اشاره می‌کند، نه بایت بعدی. یعنی آدرس به اندازهٔ `sizeof(*p)` جلو می‌رود.
- **تفریق دو اشاره‌گر** تعداد **عضوهای** بین آن‌هاست: با `p = &a[1]` و `q = &a[4]` ، `q - p = 3` .
- نام آرایه را نمی‌شود عوض کرد (`a++` خطاست)، ولی اشاره‌گر را می‌شود.
- `sizeof(a)` برای آرایه، **کل بایت‌ها** است؛ برای اشاره‌گر، ۸. پس تعداد عضوها `sizeof(a)/sizeof(a[0])` است (همان جایی که آرایه تعریف شده).

آرایهٔ دوبعدی

`int b[3][4]` سطری ذخیره می‌شود. `b[i][j]` همان `*(*(b + i) + j)` است و آدرسش `پایه + (i·4 + j)·4`

- `sizeof(b) = 48` ، `sizeof(b[0]) = 16` (یک سطر).
- اگر `p = &b[0][0]` ، آنگاه `*(p + k)` همان عضو kام در ترتیب سطری است.

درس ۳: رشته

- رشته آرایه‌ای از `char` است که با `'\0'` تمام می‌شود.
- `char s[] = "hello";` ⇐ آرایهٔ ۶ بایتی و **قابل تغییر**. `sizeof(s) = 6` و `strlen(s) = 5` .
- `char *t = "hello";` ⇐ اشاره‌گر به رشتهٔ ثابت. `sizeof(t) = 8` . تغییر `t[0]` رفتار تعریف‌نشده است.
- `strlen` تا `'\0'` می‌شمارد (بدون خودش). `sizeof` اندازهٔ آرایه را می‌دهد، هر چقدر هم پر شده باشد.
- `strcmp(a, b)` : منفی اگر a در ترتیب الفبایی قبل از b باشد، صفر اگر برابر باشند، مثبت اگر بعد باشد.
- `printf("%s", p)` از p تا `'\0'` چاپ می‌کند. پس اگر `p = s + 2` ، از کاراکتر سوم چاپ می‌شود.

```
const char *s) { const char *t = s; while (*t) t++; return t - s; }   // len("kankur") = 6
```

تمرین فصل ۵

```
= {10, 20, 30, 40};  int *p = a;
%d %d %d %d", *p + 1, *(p + 1), p[3], 2[a]);

[4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
%d %d", *(*(b + 1) + 2), *(b[2] + 1));

 = "abcdef";  char *q = s + 2;  printf("%s %c", q, *(q + 2));
```

> 🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۳: رشته)

📝 آزمون فصل ۵

> تست‌های **تألیفی**. خروجی همه با `gcc` تأیید شده است.

زمان: ۱۵ دقیقه. خروجی هر کد چیست؟

۱.

```
= {1, 2, 3, 4, 5};  int *p = a + 1;  p += 2;
%d %ld", *p, (long)(p - a));
```

۱) `3 4` ۲) `2 3` ۳) `12 4` ۴) `4 5`

۲.

```
= {5, 10, 15};  int *p = a;
1;
%d %d", a[0], *p);
```

۱) `11 5` ۲) `10 6` ۳) `6 6` ۴) `10 5`

۳.

```
 = "computer";
"%c%c%c", *s, *(s + 3), s[strlen(s) - 1]);
```

۱) `cpr` ۲) `cmr` ۳) `cpe` ۴) `cur`

.۴

```
[3] = {{1,2,3},{4,5,6}};  int *p = &m[0][0];
"%d %d", *(p + 4), m[1][0] + *(*m + 2));
```

۱) `5 7` ۲) `4 6` ۳) `5 6` ۴) `6 7`

.۵

```
int *x, int y) { *x += y; y += *x; }

2, b = 3;  f(&a, b);  printf("%d %d", a, b);
```

۱) `5 8` ۲) `5 3` ۳) `2 3` ۴) `2 8`

.۶

```
= "abcde";  p += 2;
"%s %zu", p, strlen(p));
```

۱) `cde 3` ۲) `cde 5` ۳) `c 1` ۴) `abcde 5`

.۷

```
= {10, 20, 30};  int *p = a;
"%d", *(p + 1) + *p + 1);
```

۱) `31` ۲) `50` ۳) `40` ۴) `21`

.۸ (سیستم ۶۴ بیتی)

```
0];  int *p = a;
"%zu %zu", sizeof(a) / sizeof(a[0]), sizeof(p));
```

۱) `10 40` ۲) `10 8` ۳) `40 8` ۴) `10 10`

.۹

```
10] = "ab";  strcat(s, "cd");
"%s %zu %zu", s, strlen(s), sizeof(s));
```

۱) `abcd 4 5` ۲) `abcd 4 10` ۳) `abcd 5 10` ۴) `ab 2 10`

.۱۰

```
1, y = 2;  int *p = &x, *q = &y;
+ 1;  p = q;  *p = 7;
"%d %d", x, y);
```

۱) `7 2` ۲) `3 7` ۳) `3 2` ۴) `7 7`

🔑 **جواب‌ها و راه‌حل:** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (آزمون فصل ۵)

🗂 کارت خلاصهٔ فصل ۵

- `*p++` اشاره‌گر را جلو می‌برد. `(*p)++` و `*++p` مقدار را زیاد می‌کنند.
- `a[i] = *(a+i)`. `p+1` یک **عضو** جلو می‌رود. `q - p` تعداد عضوهاست.
- دوبعدی سطری است: `*(*(b+i)+j) = b[i][j]`. `sizeof(b[0])` یک سطر است.
- `sizeof` آرایه کل بایت‌هاست، `sizeof` اشاره‌گر ۸ است. `strlen` بدون `'\0'` می‌شمارد. `char s[]` قابل ت
  `char *t = "…"` نه.

🔑 پاسخ‌نامهٔ فصل

بخش ۱: درس ۳: رشته

۱. `11 20 40 30` (`*p + 1` یعنی ۱۰ + ۱، نه عضو بعدی)

۲. `7 10`

۳. `cdef e`

بخش ۲: آزمون فصل ۵

سؤال	۱	۲	۳	۴	۵	۶	۷	۸	۹
جواب	۱	۲	۱	۱	۲	۱	۱	۲	۲

۱. p به `a[3] = 4` اشاره می‌کند و فاصله‌اش از ابتدا ۳ **عضو** است (نه ۱۲ بایت).

۲. `*p++ += 1` یعنی `*(p++) += 1`: `a[0]` می‌شود ۶ و p به `a[1]` می‌رود.

۳. `s[0] = 'c'`، `s[3] = 'p'`، و آخرین کاراکتر `'r'` است.

۴. `p + 4` در ترتیب سطری `m[1][1] = 5` است. `*(*m + 2)` همان `m[0][2] = 3` است، پس `4 + 3 = 7`.

۵. x اشاره‌گر است، پس a می‌شود ۵. y کپی است، پس b همان ۳ می‌ماند.

۶. از `'c'` تا آخر: `cde` با طول ۳.

۷. `*(p+1) = 20` و `*p + 1 = 11`. جمع ۳۱.

۸. ۱۰ عضو؛ اشاره‌گر ۸ بایت.

۹. strlen تعداد کاراکترها را تا `'\0'` می‌شمارد (۴)؛ sizeof اندازهٔ آرایه است (۱۰).

۱۰. x می‌شود `2 + 1 = 3`. بعد p به y اشاره می‌کند، پس `y = 7`.